

Reverse Engineering Report

Password-Protected Executable Analysis

University Assignment | Nour Mohamed - 202302744

Target File	PasswordValidator_protected.exe
File Size	679,424 bytes (packed) → 1,844,736 bytes (unpacked)
Protection	UPX Packing + .NET NativeAOT + Anti-Debug
Password Found	hydra2024
Hash Algorithm	SHA-256
SHA-256 Hash	3425228F4E507D52DB9187E24CF3EDA5D5BE9C15E5453930EAA3F2C60888C474
Secret Key	XK-9F-2M-7Q-L3
Tools Used	System Informer, Strings64, 7-Zip, UPX, de4dot, Ghidra, x64dbg

1. Overview

This report documents the complete reverse engineering process applied to a password-protected Windows executable. The binary was protected using multiple layers including UPX compression, .NET NativeAOT compilation, and active anti-debugging measures. The goal was to extract the password, retrieve the secret key, and reconstruct the original source code.

2. Step 1 — Extracting the Password from Memory

2.1 Running the Target Program

The program was first executed to observe its behavior. It presented a console prompt:

```
=== Simple Password Validator ===
Enter password (attempts left: 3):
```

The program allowed 3 password attempts before locking out. Without the password, the program needed to be analyzed through memory forensics.

2.2 Creating a Memory Dump with System Informer

While the program was running and waiting for input, System Informer was used to capture a full memory dump:

- Opened System Informer and located the PasswordValidator_protected.exe process
- Right-clicked the process → Create dump file → Full
- Saved the dump as: PasswordValidator_protected.exe_2026-05-02_22-10-07.dmp

2.3 Extracting Strings from the Dump

The Sysinternals Strings64 tool was used to extract all readable strings from the memory dump:

```
Strings64.exe /accepteula "PasswordValidator_protected.exe_2026-05-02_22-10-07.dmp" > alldump_strings.txt
```

The output file was opened in Notepad and searched for keywords. The following critical strings were discovered:

String Found	Value
Password	hydra2024
Success Message	[+] PASSWORD CORRECT!
Access Message	[+] Access granted to secure data.
Secret Key	[+] Secret key: XK-9F-2M-7Q-L3
Failure Message	[-] Wrong password. Try again.
Anti-Debug List	dnspy, x64dbg, IDA pro, Cheat Engine, de4dot, ILSpy, Ghidra...

Additionally, the SHA-256 hash of the password was found at line 24741 of the strings file:

```
3425228F4E507D52DB9187E24CF3EDA5D5BE9C15E5453930EAA3F2C60888C474
```

2.4 Verifying the Password

The SHA-256 hash was independently verified by hashing hydra2024:

```
echo -n "hydra2024" | sha256sum
3425228f4e507d52db9187e24cf3eda5d5be9c15e5453930eaa3f2c60888c474  ✓ MATCH
```

The program was then run and hydra2024 was entered as the password, confirming:

```
[+] PASSWORD CORRECT!
[+] Access granted to secure data.
[+] Secret key: XK-9F-2M-7Q-L3
```

3. Step 2 — Analyzing the Binary Structure

3.1 Identifying UPX Packing

Strings64 was run directly on the executable (not the dump). The output revealed UPX section names:

```
UPX0
UPX1
UPX!
```

This confirmed the binary was compressed using UPX (Ultimate Packer for eXecutables). UPX packing compresses the original code and adds a small stub that decompresses it at runtime, making static analysis difficult.

3.2 Attempting de4dot Deobfuscation

de4dot was initially tried to deobfuscate the binary assuming it was a standard .NET assembly:

```
de4dot.exe "PasswordValidator_protected.exe"
```

Result: System.BadImageFormatException — the file does not contain .NET managed metadata. This confirmed the binary is NOT standard .NET IL bytecode but rather NativeAOT compiled code.

3.3 Unpacking with UPX

UPX was used to unpack the compressed binary:

```
upx.exe -d "PasswordValidator_protected.exe" -o "unpacked.exe"
1844736 <- 679424   36.83%   win64/pe   unpacked.exe
Unpacked 1 file.
```

The binary expanded from 679KB to 1.84MB, revealing the actual executable code for analysis.

4. Step 3 — Bypassing Anti-Debug Protection

4.1 Anti-Debug Detection

When attempting to load the unpacked.exe in debuggers (dnSpy, ILSpy, x64dbg), the program immediately terminated with:

```
Process terminated. Environment.FailFast was called
```

The strings dump revealed a comprehensive list of tools the program actively detects and blocks:

- dnspy, x64dbg, x32dbg, IDA Pro, IDA64, idaq, idaq64
- Cheat Engine, cheatengine-x86_64
- de4dot, ILSpy, dotPeek, JetBrains Rider
- OllyDbg, Immunity Debugger, WinDbg
- Wireshark, Fiddler, Process Monitor, Process Hacker
- Ghidra (indirect detection)

4.2 Finding the Anti-Debug Function in Ghidra

Ghidra was used to analyze the unpacked binary. The anti-debug checking function was located by:

- Searching Defined Strings for 'x64dbg' → found at address 0x1401a4d3f
- Following the XREF to function FUN_14000d910
- Identifying the key check: TEST AL,AL / JZ LAB_14000d941

4.3 Patching the Anti-Debug Check

The anti-debug check was patched directly in Ghidra:

Address	0x14000d932
Original Instruction	TEST AL,AL (bytes: 84 C0)
Patched Instruction	XOR AL,AL (bytes: 30 C0)
Effect	Forces AL=0, making JZ always jump to safe code path

The patched binary was exported as patched.exe using File → Export Program → Original File. Verification: patched.exe was run successfully, showing the password prompt without crashing, confirming the anti-debug was fully disabled.

5. Step 4 — Decompiling and Source Code Reconstruction

5.1 Loading in Ghidra

The unpacked.exe was imported into Ghidra and analyzed. The binary structure revealed:

- .text — executable code section
- .managed — .NET NativeAOT metadata
- .hydrated — runtime data
- .rdata — read-only data
- .data — initialized data

5.2 Navigating the Call Chain

The main password validation function was located by following the call chain:

entry()	Program entry point
FUN_140061240()	CRT startup / scrt_common_main
FUN_140002560()	Main dispatcher
FUN_1400e14e0()	PASSWORD VALIDATION FUNCTION ← KEY

5.3 Analyzing the Password Validation Function

The decompiled code of FUN_1400e14e0 revealed the complete password validation logic:

- The function runs in a loop with 3 attempts (*(IVar3+8) = 3, then 2, then 1)
- FUN_14006af90() — reads user input from console
- FUN_1400e16b0() — computes SHA-256 hash of the input
- DAT_140127610 — stores the correct SHA-256 hash (hydra2024)
- FUN_1400788f0() — compares the computed hash with the stored hash
- On match: prints success messages and secret key
- On failure: prints wrong password message, decrements attempts

5.4 SHA-256 Hash Function Analysis

FUN_1400e16b0 was identified as the SHA-256 hashing function by:

- Finding SHA256 string reference at 0x14017d8c2
- The function creates a SHA256 hash object, processes the input, and returns hex string
- Uses .NET NativeAOT SHA256 implementation (BCrypt provider)

5.5 String Comparison Analysis

FUN_1400788f0 performs Unicode string comparison:

- Compares string lengths first (*(int*)(param_1+8) == *(int*)(param_2+8))
- Then calls FUN_140085880 — a vectorized memcmp using SIMD instructions
- Processes 16 bytes at a time for performance
- Returns 1 on match, 0 on mismatch

6. Reconstructed Source Code

Based on the complete Ghidra analysis, the original C# source code was reconstructed:

```
// Reconstructed C# Source Code
using System;
using System.Security.Cryptography;
using System.Text;

class PasswordValidator
{
    static void Main()
    {
        // DAT_140127610 - stored SHA-256 hash
        string correctHash =
            "3425228F4E507D52DB9187E24CF3EDA5D5BE9C15E5453930EAA3F2C60888C474";
        string secretKey = "XK-9F-2M-7Q-L3";

        int attempts = 3; // *(lVar3+8) = 3
    }
}
```

```
Console.WriteLine("=== Simple Password Validator ===");

while (attempts > 0) // FUN_1400e14e0 loop
{
    // FUN_14006af90 - reads console input
    Console.Write($"Enter password (attempts left: {attempts}): ");
    string input = Console.ReadLine();

    // FUN_1400e16b0 - SHA-256 hash function
    string inputHash = ComputeSHA256(input);

    // FUN_1400788f0 - string comparison
    if (inputHash.Equals(correctHash, StringComparison.OrdinalIgnoreCase))
    {
        Console.WriteLine("[+] PASSWORD CORRECT!"); // DAT_1401261c0
        Console.WriteLine("[+] Access granted to secure data.");
        Console.WriteLine($"[+] Secret key: {secretKey}"); //
DAT_14011fbb0
        return;
    }
    attempts--;
    if (attempts > 0)
        Console.WriteLine("[-] Wrong password. Try again."); //
DAT_140126348
}
Console.WriteLine("[-] Maximum attempts exceeded. Access denied.");
}

// FUN_1400e16b0 - SHA-256 implementation
static string ComputeSHA256(string input)
{
    using var sha256 = SHA256.Create();
    byte[] bytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(input));
    return Convert.ToHexString(bytes);
}
}
```

7. Summary of Findings

Finding	Details
RAR Password	hydra2024 (found via memory dump)
Program Password	hydra2024
Hash Algorithm	SHA-256
Hash Value	3425228F4E507D52...EAA3F2C60888C474
Secret Key	XK-9F-2M-7Q-L3
Packer	UPX 4.24
Compiler	Microsoft .NET NativeAOT
Anti-Debug	Environment.FailFast + 20+ tool detection

Patch Applied	TEST AL,AL → XOR AL,AL at 0x14000d932
Key Functions	FUN_1400e14e0 (main), FUN_1400e16b0 (SHA256), FUN_1400788f0 (compare)

8. Tools Used

Tool	Purpose
System Informer	Process memory dump creation
Strings64 (Sysinternals)	Extract readable strings from memory dump
7-Zip	Extract RAR archive
de4dot	Attempted .NET deobfuscation (confirmed NativeAOT)
UPX 5.1.1	Unpack UPX-compressed binary
Ghidra 12.0.4	Static analysis and decompilation
x64dbg	Dynamic debugging of patched binary
Python 3	Memory scanning scripts for live process analysis

9. Conclusion

The password-protected executable was successfully analyzed through a multi-stage reverse engineering process. Despite multiple layers of protection including UPX packing, .NET NativeAOT compilation, and active anti-debugging measures, the following was achieved:

- Password hydra2024 was extracted from memory using System Informer and Strings64
- The SHA-256 hash 3425228F... was identified and verified
- The UPX packing layer was removed using UPX -d
- The anti-debug protection was patched in Ghidra (TEST AL,AL → XOR AL,AL)
- The main password validation function FUN_1400e14e0 was fully analyzed
- The complete source code was reconstructed with all function addresses mapped
- The secret key XK-9F-2M-7Q-L3 was successfully retrieved

Note: Complete decompilation back to original C# source is not possible with NativeAOT binaries as the IL bytecode is compiled directly to native machine code. The reconstructed source code above represents the accurate functional equivalent verified through both static and dynamic analysis.